

Stealth and Beyond: Attribute-Driven Accountability in Bitcoin Transactions

Alberto Maria Mongardini^{1,2*}, Daniele Friolo^{1*}, and Giuseppe Ateniese³

¹ Department of Computer Science, Sapienza University of Rome, Rome, Italy
{mongardini, friolo}@di.uniroma1.it

² Department of Applied Mathematics and Computer Science, Technical University of Denmark, Copenhagen, Denmark
among@dtu.dk

³ Department of Computer Science, George Mason University, Fairfax, VA, USA
ateniese@gmu.edu

Abstract. Bitcoin enables decentralized, pseudonymous transactions, but balancing privacy with accountability remains a challenge. This paper introduces a novel dual accountability mechanism that enforces both sender and recipient compliance in Bitcoin transactions. Senders are restricted to spending Unspent Transaction Outputs (UTXOs) that meet specific criteria, while recipients must satisfy legal and ethical requirements before receiving funds. We enhance stealth addresses by integrating compliance attributes, preserving privacy while ensuring policy adherence. Our solution introduces a new cryptographic primitive, Identity-Based Matchmaking Stealth Signatures (IB-MSS), which supports streamlined auditing. Our approach is fully compatible with existing Bitcoin infrastructure and does not require changes to the core protocol, preserving both privacy and decentralization while enabling transaction auditing and compliance.

Keywords: Stealth addresses · Accountability · Bitcoin

1 Introduction

Bitcoin has transformed financial transactions by enabling pseudonymous, decentralized exchanges without intermediaries. This model empowers users to manage assets autonomously and conduct global transactions freely. However, as cryptocurrency adoption grows, so too does the tension between *privacy* and *regulatory compliance*—particularly regarding Anti-Money Laundering (AML) and Know Your Customer (KYC) requirements. While Bitcoin offers pseudonymity, its transparent public ledger exposes transaction histories to public scrutiny, making it increasingly difficult to preserve user privacy while adhering to legal standards.

* A. M. Mongardini, D. Friolo — The work was carried out while the authors were visiting George Mason University, Fairfax, VA, USA. The two authors contributed equally.

To address privacy concerns, *stealth addresses* [13,26,32] were introduced. These allow recipients to generate one-time-use addresses that are not directly linked to their public Bitcoin addresses. This obfuscation ensures that third parties cannot easily trace the recipient of a transaction, providing a layer of privacy by breaking the linkability of transactions. Stealth addresses thus help preserve recipient anonymity within Bitcoin’s inherently transparent system.

However, while stealth addresses enhance privacy, they fall short in addressing the equally critical need for *accountability*. They do not provide mechanisms to ensure that transactions comply with legal or ethical standards, nor can they enforce policy-based criteria in complex regulatory environments. For example:

- **Donations to organizations in restricted regions:** NGOs operating in politically sensitive regions may need to ensure that donations come from legitimate, legally compliant sources to avoid accepting funding/donations from illicit entities. Traditional stealth addresses ensure donor privacy but lack the ability to verify that the funds originate from vetted and legally approved sources. This leaves the NGO vulnerable to unknowingly accepting illegal donations, exposing it to serious legal risks.
- **Compliance in cross-border remittances:** Individuals sending remittances across borders often face regulatory scrutiny, especially in cases involving countries under sanctions or financial restrictions. For example, sending funds to a sanctioned country could violate international regulations. While stealth addresses can maintain privacy by hiding recipient details, they provide no way to enforce compliance with such regulations, potentially leading to unlawful transactions.
- **Ensuring AML compliance for cryptocurrency exchanges:** Cryptocurrency exchanges must comply with AML/KYC regulations, which require them to validate the identities of both senders and recipients. Traditional stealth addresses obscure participants for privacy, but do not allow for built-in verification of AML/KYC compliance, leaving exchanges struggling to balance regulatory adherence with user privacy.
- **Private investments in regulated markets:** Investors participating in private transactions, such as venture capital investments or equity crowdfunding, often require anonymity for various reasons (*e.g.*, protecting strategic financial interests). However, these transactions must comply with securities laws, which require verification of accredited investor status or legal financial limits. Traditional stealth addresses provide anonymity but lack mechanisms to ensure that only accredited or verified investors participate in these transactions.

In response to these challenges, we propose an enhanced form of stealth addresses that goes beyond privacy protection to incorporate accountability. Our solution integrates a novel cryptographic primitive (*Identity-Based Matchmaking Stealth Signatures*) into stealth addresses, embedding compliance attributes directly into the address-derivation process. This ensures that both *privacy* and *accountability* are preserved.

Specifically, our approach introduces a *dual accountability* mechanism:

1. **Sender Accountability:** The recipient can only spend funds if the sender has met specific criteria, such as originating from a permitted country or complying with AML regulations.
2. **Recipient Accountability:** The sender can ensure that the recipient satisfies legal or ethical requirements, preventing the transfer of funds to sanctioned or restricted entities.

Our solution integrates seamlessly with the existing Bitcoin infrastructure, requiring no changes to the Bitcoin Core protocol. The only addition is a *Certification Authority (CA)*, which issues compliance certificates to users, verifying that both senders and recipients meet the necessary standards. The CA can be implemented in centralized or decentralized formats, depending on the regulatory context or user preference.

Our contributions. This paper introduces a comprehensive framework for enhancing accountability in Bitcoin transactions through the use of *stealth addresses with attributes*, underpinned by a novel cryptographic primitive, *Identity-Based Matchmaking Stealth Signatures (IB-MSS)*. Our key contributions are as follows:

- We introduce the *IB-MSS* primitive, which integrates dual accountability attributes into stealth addresses for both the sender and the recipient. Inspired by Matchmaking Encryption (ME) [5], where decryption requires both parties’ attributes to satisfy each other’s policies, our IB-MSS adapts this concept to signatures. This enables signature verification only when both the sender’s and recipient’s attributes fulfill each other’s specified policies, ensuring dual accountability.
Building upon non-interactive key distribution [21] and digital signature schemes, we present the detailed construction of IB-MSS in [Section 5](#).
- We explore the integration of a *Certification Authority (CA)* to validate sender and recipient attributes. We discuss centralized and decentralized implementations of this authority in [Section 5.3](#).
- We provide, in [Section 6](#), an implementation of stealth addresses with attributes using IB-MSS and benchmark its performance. Our implementation is inspired by Cerulli *et al.*’s work on verifiably encrypted threshold key derivation [9]. We further discuss practical considerations of our implementation in [Section 7](#).

Moreover, in [Section 2.1](#), we also introduce potential enhancements, such as time-locked stealth addresses and stealth transactions with embedded messages.

1.1 Notation

Let $\mathbb{N}$ be the set of all natural numbers; for $n \in \mathbb{N}$, we write $[n]$ to denote the set $\{1, \dots, n\}$. We use calligraphic letters to denote sets (such as $\mathcal{X}$). Throughout the paper, we use the abbreviation PPT to refer to probabilistic polynomial

time. Given a PPT algorithm A , we denote by $A(x)$ the probability distribution of the output of A when run on input x . If algorithm A has oracle access to a set of oracles $\mathcal{O}$, we denote by $\mathcal{Q}_O$ the ordered list (transcript) of oracle interactions for an oracle $O \in \mathcal{O}$; $\mathcal{Q}_O[i]$ denotes the i -th transcript entry, and $x \in \mathcal{Q}_O$ means that x appears in the transcript. We use $\mathcal{Q}_O := \mathcal{Q}_O \cup \{x\}$ as shorthand for appending an entry x to the transcript.

We use the symbol $:=$ to assign the output value of the algorithm on the right-hand side to the variable on the left-hand side. When x is chosen uniformly from a set $\mathcal{X}$, we write $x \leftarrow \mathcal{X}$. If A is an algorithm, we write $y \leftarrow A(x)$ to denote a run of A on input x with output y and random tape $r \leftarrow \{0, 1\}^\lambda$.

We denote the security parameter by $\lambda \in \mathbb{N}$, and an arbitrary positive polynomial by $\text{poly}(\cdot)$. Every algorithm takes the security parameter λ as input (in unary, i.e., 1^λ). When an algorithm has multiple inputs, 1^λ is typically omitted. A function $\nu : \mathbb{N} \rightarrow \mathbb{R}$ is called negligible if, for every positive polynomial $\text{poly}(\cdot)$ and for all sufficiently large λ , it holds that $\nu(\lambda) \leq \frac{1}{\text{poly}(\lambda)}$. We use $\text{negl}(\lambda)$ to denote an unspecified negligible function. In the remainder of the paper, we consider a type-1 pairing $e : \mathbb{G}_1 \times \mathbb{G}_1 \rightarrow \mathbb{G}_2$.

2 Stealth addresses

Bitcoin's public ledger makes all transactions visible, exposing user activity to analysis. This compromises the pseudonymity of traditional addresses, as they can be traced back to individuals. Stealth addresses counter this by breaking the visible link between sender and recipient.

Basic Stealth Address Protocol (BSAP) Introduced in 2011 [13], stealth addresses create temporary, one-time-use public addresses to enhance transaction privacy. For simplicity, we use multiplicative notation. Let $\mathbb{G}$ be a group of prime order p with generator g . Alice (the sender) and Bob (the receiver) have public/secret key pairs $(\text{pk}_A, \text{sk}_A)$ and $(\text{pk}_B, \text{sk}_B)$, where $\text{pk}_A = g^a$, $\text{pk}_B = g^b$, $\text{sk}_A = a$, and $\text{sk}_B = b$ for $a, b \leftarrow \mathbb{Z}_p$. Using the Diffie-Hellman protocol, they establish a shared secret k :

- $k = H(g^{\text{sk}_A \text{sk}_B}) = H(g^{ab}) = H(\text{pk}_B^a) = H(\text{pk}_A^b)$,
- Alice uses g^k as the recipient's address,
- Bob spends funds sent to g^k using k as his private key.

While this protocol provides privacy by generating one-time-use addresses, the static nature of the address reduces privacy over multiple transactions. Both parties also share the ability to compute the private key, which limits security.

Improved Stealth Address Protocol (ISAP) Proposed in 2013 [26] and adopted in Bitcoin in 2014 [32], ISAP improves upon BSAP by introducing more dynamic addresses:

- Alice generates a fresh key pair (r, R) , where $R = g^r$ and $r \leftarrow \mathbb{Z}_p$. She computes $k = H(\text{pk}_B^r) = H(g^{r^b}) = H(R^b)$,
- She computes $Q = g^k \cdot \text{pk}_B$ and creates a transaction with Q and embeds R (either in `OP_RETURN` or as part of the transaction signature’s randomness [36]) to ensure stealthiness,
- Bob monitors the blockchain for transactions containing R' , computes $k' = H(R'^b)$, and reconstructs the recipient address $Q' = g^{k'} \cdot \text{pk}_B$. If $Q' = Q$, he can spend the transaction using $k' + b$.

While ISAP improves privacy by creating fresh addresses, Bob must use his private spending key to monitor the blockchain, which introduces a security risk. Although Bob does not directly share his private spending key with any third party, actively using it for blockchain scanning increases the risk of exposure. Indeed, repeated use of his sensitive key makes it vulnerable to potential threats such as software bugs, memory leaks, or malware.

Dual Key Stealth Address Protocol (DKSAP) DKSAP mitigates ISAP’s security concerns by introducing two separate key pairs for tracking and spending:

- Bob holds two key pairs: a tracking key pair $(\text{ptk}_B, \text{stk}_B)$ with $\text{stk}_B = s$ and $\text{ptk}_B = g^s$, and a spending key pair $(\text{pk}_B, \text{sk}_B)$,
- Alice generates (r, R) and computes $k = H(\text{ptk}_B^r)$,
- She calculates $Q = g^k \cdot \text{pk}_B$ and creates a transaction with Q and R ,
- Bob monitors the blockchain using the tracking key stk_B to find R' , computes $k' = H(R'^s)$, and reconstructs $Q' = g^{k'} \cdot \text{pk}_B$. If $Q' = Q$, Bob can spend the transaction using the derived one-time secret key $k' + b$.

By separating the tracking and spending keys, Bob can outsource transaction monitoring without compromising his private spending key, enhancing both security and privacy.

2.1 Enhancements of Stealth Addresses

While stealth addresses enhance privacy in Bitcoin, our work extends their functionality with additional cryptographic features. In this subsection, we introduce our enhancements to stealth addresses, which serve as an intermediate step toward presenting our main contribution. We integrate concepts such as Identity-Based Encryption (IBE) and time-lock mechanisms, enabling the inclusion of hidden information within stealth addresses to support more advanced use cases.

One practical application is the creation of time-locked stealth transactions, where cryptographic puzzles are embedded in the shared key k . These puzzles ensure that the recipient can only spend the transaction after a certain time has passed. For example:

$$k = H(\text{pk}_B^r, \text{"101101"}) = H(g^{br}, \text{"101101"}).$$

where pk_B is the recipient's public key, r is a random value chosen by the sender, and g^{br} is the Diffie-Hellman shared key. The value "101101" serves as a time-lock condition.

Unlike traditional time-lock methods, where the lock time is visible, this approach embeds the time constraint within the stealth address. Only the recipient, who computes k , can identify the time-lock, preserving privacy while preventing censorship due to visible lock times.

To further refine time control, we can use a puzzle based on repeated squaring [25]:

$$H(\text{pk}_B^r, 2^e) = H(g^{br}, 2^e),$$

where $e = 2^t \bmod \phi(n)$, with t representing the desired lock time, and $n = pq$ being the product of two large primes p and q chosen by Alice. This ensures that only after time t can the recipient unlock and spend the funds.

The same cryptographic puzzle string used in time-locked addresses can double as a covert message, ensuring confidentiality. This dual-use of k for time-locking and discreet communication allows only the intended recipient to decode and access the message, such as "42":

$$k = H(\text{pk}_B^r, "42") = H(g^{br}, "42").$$

Incorporating IBE into k adds further flexibility by allowing identity-specific transactions. In IBE, a sender encrypts data so that only a recipient matching a specific identity (e.g., an email address) can decrypt it. The receiver needs a decryption key tied to their identity, issued by a trusted authority.

In the Boneh-Franklin IBE scheme [7], the trusted authority generates a decryption key for the identity id as $H'(\text{id})^s$, where s is the authority's secret key. The sender, knowing the recipient's identity id and the authority's master public key $\text{mpk} = g^s$, encrypts the message as follows:

$$c = (g^t, m \oplus e(H'(\text{id}), g^s)^t),$$

where t is a random value. The recipient, with the decryption key $H'(\text{id})^s$, can retrieve the message m by using the pairing properties:

$$e(H'(\text{id})^s, g^t) = e(H'(\text{id}), g^s)^t.$$

By adapting this IBE structure, the shared key k can be modified as follows:

$$k = H(\text{pk}_B^r, e(H'(\text{id}), g^s)^r) = H(g^{br}, e(H'(\text{id})^s, g^r)).$$

This adaptation allows the sender to restrict the transaction to a specific recipient identity, providing additional security and control. In the context of IBE, the identity id can represent not just simple identifiers like email addresses, but also complex attributes or conditions required for the recipient to access the funds. For example, id could be defined as $\text{id} = \text{"manager"} \wedge \text{"after January 2028"}$,

meaning the recipient must be a manager and can only access the funds after January 2028. In this case, the trusted authority issues the decryption key only when both conditions are met. This attribute-based approach extends the flexibility of stealth addresses, enabling conditional access based on roles, time constraints, or other attributes, making them suitable for sophisticated use cases.

This solution requires no changes to Bitcoin Core or transaction structure. The recipient runs a program, similar to current stealth address handling, to identify transactions they can decrypt. The only addition is an external authority for issuing decryption keys when the conditions are satisfied. This ensures enhanced privacy and accountability without disrupting standard transaction processes.

3 Related Works

Considerable progress has been made in formalizing stealth addresses and implementing efficient solutions for Bitcoin and Ethereum.

Formalization. Stealth addresses have seen significant formalization efforts, beginning with the work of Fleischhacker *et al.* [12], who introduced Digital Signatures with Re-Randomizable Keys. This model established a strong unforgeability notion, allowing adversaries to query a key re-randomization oracle while maintaining security.

Meiklejohn and Mercer [18] proposed stealth keys, where a shared secret and nonce allow one-time public and private key derivation, with indistinguishability guarantees between derived and freshly generated keys. Fauzi *et al.* [10] extended this model to support Updatable Public Keys, ensuring that public keys could be refreshed without revealing whether they were newly generated or updated, further strengthening the privacy of stealth addresses.

Backes *et al.* [6] introduced Signatures with Flexible Keys, allowing users to transform public keys into equivalent forms, while maintaining indistinguishability, even when an adversary has access to the randomness used in key generation.

Liu *et al.* [15] identified a vulnerability in deterministic wallets that could expose the master secret key if a single one-time key were compromised. They proposed Key-Insulated and Privacy-Preserving Signatures (PDPKS) to address key exposure risks. This work was extended by Pu *et al.* [24], who introduced Stealth Signatures, providing stronger protection against key exposure and offering a post-quantum secure construction. Our IB-MSS design builds on these advancements, particularly the formalization of stealth addresses.

A recent approach to enhance recipient privacy is the Silent Payments protocol [14]. It operates similarly to stealth addresses but differs in how the one-time-use address is constructed. Instead of including an explicit nonce R (as in ISAP), Alice derives the one-time-use address using the private key corresponding to one of the UTXOs spent in the payment. This removes the need for R while keeping the stealth address dynamic. While Silent Payments remove the need for R , the recipient still needs to monitor the blockchain to identify incoming transactions. Like the original stealth address protocols, Silent Payments use the

Diffie-Hellman protocol. Thus, the enhancements we introduce in [Section 2.1](#), as well as the Accountable Stealth Addresses with Attributes protocol ([Section 5](#)), can also be applied to Silent Payments.

Implementation. The development of stealth addresses for blockchains has also progressed. Wahrstätter *et al.* [35] introduced the BaseSap protocol, designed for programmable blockchains, using the secp256k1 elliptic curve. Their implementation leverages view tags to optimize transaction parsing, enabling more efficient handling of stealth addresses in decentralized applications.

Recent privacy-focused efforts, such as Privacy Pools by Buterin *et al.* [8], proposed in response to sanctions on Tornado Cash [33], further highlight the need for privacy-preserving systems that allow users to prove compliance with legal frameworks. Privacy Pools aim to enable users to demonstrate that their funds are not connected to illicit activities while maintaining privacy. However, challenges in preventing illegal transactions through privacy tools remain [28].

Privacy Models in Monero and Zcash. The concept of breaking the link between a recipient’s identity and on-chain transactions is central to privacy-focused cryptocurrencies like Monero and Zcash, though with different approaches and guarantees than our work. Monero implements the Dual-Key Stealth Address Protocol (DKSAP) [19], which we describe in [Section 2](#). DKSAP separates tracking and spending keys, enabling unlinkability between transactions while protecting the receiver’s privacy. However, Monero’s model is purely privacy-centric: it provides no mechanism to enforce attribute-based policies or verify sender/recipient compliance with regulatory requirements. Conversely, Zcash takes a fundamentally different approach using Zero-Knowledge Succinct Non-Interactive Argument of Knowledge (zk-SNARKs) [30]. Rather than deriving one-time addresses through key derivation (as in stealth addresses), Zcash encrypts all transaction metadata (sender, receiver, and amount) and publishes only a zero-knowledge proof of transaction validity on the ledger. This provides stronger confidentiality by completely obscuring the transaction graph. However, like Monero, Zcash does not support selective accountability or policy enforcement based on participant attributes.

Our IB-MSS scheme extends the stealth address paradigm in a fundamentally different direction. While maintaining the core privacy guarantees of stealth addresses, we introduce dual accountability. By cryptographically binding address derivation to compliance attributes for both sender and recipient, we ensure that a transaction can only be spent when both parties satisfy each other’s policy requirements.

4 Preliminaries

In the following, we will introduce the two main building blocks for constructing our IB-MSS.

4.1 Identity-Based Non-Interactive Key Distribution

Identity-Based Non-Interactive Key Distribution was initially introduced by Paterson and Srinivasan [21].

Recall that in a key distribution scheme, Alice distributes a secret key to Bob over a public channel without letting any eavesdropper learn the shared key.

When a key distribution scheme is non-interactive, Alice and Bob can compute the same secret without exchanging messages with each other.

In the Identity-Based variant, an authority can generate retrieval keys associated with the parties' identities and send them to the corresponding parties. Alice and Bob can now obtain the same shared key by using their own retrieval key and the identity of the other party, *i.e.*, Alice uses her own retrieval key associated with the string "Alice" together with the string "Bob" to compute the shared key k , and, on the other hand, Bob uses his own retrieval key associated with the string "Bob" together with the string "Alice" to obtain the very same key k . In the context of Identity-Based Non-Interactive Key Distribution, the only interaction is the one-time distribution of retrieval keys from the authority to the parties; no further communication with each other or with the authority is required. The formal description of an IB-NIKD follows.

Definition 1 (IB-NIKD). *Let $\mathcal{SHK}$ be a shared key space. An IB-NIKD scheme Π is a tuple of algorithms (Setup, Extract, SharedKey). The algorithms are described as follows:*

Setup(1^λ): *On input the security parameter, this randomized algorithm outputs a master public key mpk and a master secret key msk .*

Extract($\text{mpk}, \text{msk}, \text{id}$): *On input the master public key mpk , the master secret key msk , and an identifier $\text{id} \in \{0, 1\}^*$, this deterministic algorithm outputs a retrieval key rk_{id} .*

SharedKey($\text{mpk}, \text{rk}_{\text{id}_A}, \text{id}_B$): *On input the master public key mpk , a retrieval key rk_{id_A} , and an identifier $\text{id}_B \in \{0, 1\}^*$, this deterministic algorithm outputs a shared key $k \in \mathcal{SHK}$.*

Definition 2 (Correctness). *An IB-NIKD is correct if $\forall \text{id}_A, \text{id}_B \in \{0, 1\}^*$,*

$$\Pr[\text{SharedKey}(\text{mpk}, \text{rk}_{\text{id}_A}, \text{id}_B) = \text{SharedKey}(\text{mpk}, \text{rk}_{\text{id}_B}, \text{id}_A)] = 1,$$

where $(\text{mpk}, \text{msk}) \leftarrow^ \text{Setup}(1^\lambda)$, $\text{rk}_{\text{id}_A} := \text{Extract}(\text{mpk}, \text{msk}, \text{id}_A)$, and $\text{rk}_{\text{id}_B} := \text{Extract}(\text{mpk}, \text{msk}, \text{id}_B)$.*

Definition 3 (IND-Security). *An IB-NIKD scheme Π is IND-secure if, for all valid PPT adversaries $\mathcal{A}$, $\Pr[\text{IND-SK}_\mathcal{A}^\Pi(\lambda) = 1] \leq \frac{1}{2} + \text{negl}(\lambda)$.*

Let $\mathcal{O} = \{\text{Extract}(\text{mpk}, \text{msk}, \cdot), \text{Reveal}\mathcal{O}\}$.

The adversary $\mathcal{A}$ is valid if, letting $(\text{id}_A, \text{id}_B, \text{st})$ be the output of $\mathcal{A}^\mathcal{O}(\text{mpk})$ in the experiment, for all $\text{id}^ \in \mathcal{Q}_{\text{Extract}(\text{mpk}, \text{msk}, \cdot)}$ we have $\text{id}^* \neq \text{id}_A \wedge \text{id}^* \neq \text{id}_B$, and $(\text{id}_A, \text{id}_B) \notin \mathcal{Q}_{\text{Reveal}\mathcal{O}}$ and $(\text{id}_B, \text{id}_A) \notin \mathcal{Q}_{\text{Reveal}\mathcal{O}}$.*

The experiment $\text{IND-SK}_\mathcal{A}^\Pi(\lambda)$ is described in Fig. 1.

IND-SK _A ^{II} (λ)	RevealO(id _A [*] , id _B [*])
(mpk, msk) ← _{\$} Setup(1 ^λ)	rk _{id_A[*]} := Extract(mpk, msk, id _A [*])
(id _A , id _B , st) ← _{\$} A ^O (mpk)	return SharedKey(mpk, rk _{id_A[*]} , id _B [*])
b ← _{\$} {0, 1}	
rk _{id_A} := Extract(mpk, msk, id _A)	
if b = 0	
k := SharedKey(mpk, rk _{id_A} , id _B)	
else	
k ← _{\$} SHK	
b' ← _{\$} A ^O (mpk, id _A , id _B , k, st)	
return b $\stackrel{?}{=}$ b'	

Fig. 1. The IB-NIKD indistinguishability experiment IND-SK_A^{II}(λ). We implicitly assume that, other than the adversarial inputs, oracles have access to the internal state of the main experiment.

4.2 Signature Schemes

Definition 4 (Signatures). A signature scheme is a tuple of algorithms $\Pi = (\text{KGen}, \text{Sign}, \text{Vrfy})$ with message space $\mathcal{M}$, described as follows:

KGen(1^λ): On input the security parameter, this randomized algorithm outputs a public/secret key pair (pk, sk).

Sign(sk, m): On input a signing key sk and a message $m \in \mathcal{M}$, this randomized algorithm outputs a signature τ .

Vrfy(pk, m, τ): On input a public key pk and a message $m \in \mathcal{M}$, this deterministic algorithm outputs 1 if the signature verifies, and 0 otherwise.

Definition 5 (Correctness). A signature scheme Π is correct if, for all $m \in \mathcal{M}$, $\Pr[\text{Vrfy}(\text{pk}, m, \tau) = 1] = 1$, where $(\text{pk}, \text{sk}) \leftarrow \text{KGen}(1^\lambda)$ and $\tau \leftarrow \text{Sign}(\text{sk}, m)$.

Definition 6 (EUF-CMA). A signature scheme Π is existentially unforgeable under chosen message attacks if, for all PPT adversaries $\mathcal{A}$,

$$\Pr \left[\text{Vrfy}(\text{pk}, m, \tau) = 1 \wedge m \notin \mathcal{Q}_{\text{Sign}(\text{sk}, \cdot)} \mid \begin{array}{l} (\text{pk}, \text{sk}) \leftarrow \text{KGen}(1^\lambda); \\ (m, \tau) \leftarrow \text{A}^{\text{Sign}(\text{sk}, \cdot)}(\text{pk}) \end{array} \right] \leq \text{negl}(\lambda).$$

5 Accountable Stealth Addresses with Attributes

Stealth addresses protect identities but cannot enforce *who* may pay *whom*. We close that gap by folding a short *attribute certificate*—for instance “citizen of X ”—directly into the Diffie–Hellman-style secret that a stealth address already uses. No extra bytes appear on-chain; the policy is evaluated inside the shared secret k .

Running example. Bob accepts coins only from citizens of X , whereas Alice will pay only citizens of Y . A certification authority (CA) issues long-lived credentials $C_X = H_1(X)^s$ and $C_Y = H_1(Y)^s$, where s is the CA’s master secret and H_1 hashes its input into the pairing group.

Creating the payment. Before funding the output, Alice computes

$$k := H_2(e(C_X, H_1(Y)))$$

and derives a one-time public key from k ; the resulting script is an ordinary SegWit OP_0 <20-byte hash> (P2WPKH).

Spending it. When Bob scans the chain, he evaluates

$$k' := H_2(e(H_1(X), C_Y)).$$

The two values coincide only if *both* policies hold; otherwise, Bob derives a different key and cannot detect or spend the output.

Discouraging credential resale. A raw certificate such as $H_1(\text{citizen_of } X)^s$ is portable. To curb resale, the CA can apply one of two *off-chain* restrictions that leave the on-chain script unchanged:

- **Hardware anchoring.** The certificate is sealed in a “no-export” secure element (*e.g.*, FIDO2 token [17], Trusted Platform Module (TPM) [22], or phone enclave [4,29]). The chip may also store a private **NotAfter** date and/or a small usage counter; once either limit is reached, it refuses further calls. Reselling, therefore, requires handing over the physical token, which stops working after its internal quota or deadline.
- **Optional time-window rotation.** If hardware anchoring is unavailable, the CA can simply re-issue the common “citizen-of- X ” certificate to certified users on a fixed schedule (*e.g.*, once per month). A leaked certificate then expires automatically when the window closes. This measure *only shortens the resale lifetime*; the original owner could still leak the next month’s key, so it should be viewed as a stop-gap rather than a complete anti-resale solution.

Formalization. The scheme instantiates an *Identity-Based Matchmaking Stealth Signature* (IB-MSS), adapted from the stealth-signature framework of Pu *et al.* [24]. IB-MSS provides correctness, unlinkability and unforgeability. The on-chain transaction remains byte-for-byte identical to a standard SegWit P2WPKH output.

5.1 Identity-Based Matchmaking Stealth Signatures

We define a new primitive called Identity-Based Matchmaking Stealth Signatures (IB-MSS). In IB-MSS, a sender generates a one-time public key using a retrieval key related to their identity, obtained from a Private Key Generator (PKG), and coupled with the receiver’s identity. Symmetrically, the receiver can recover the one-time secret key related to the same one-time public key by using their

retrieval key, also obtained from the PKG, and coupled with the sender's identity. The formalization of this concept is inspired by prior work on Matchmaking Encryption by Ateniese *et al.* [5]. We follow a similar algorithmic structure.

We formalize two main properties: unforgeability and unlinkability. Unforgeability ensures that no external adversary can forge a signature verifiable from such a one-time public key. Unlinkability ensures that no external adversary can distinguish between a public key generated from a standard digital signature scheme's key generation algorithm and a one-time public key generated from the IB-MSS, even if the respective one-time secret key is leaked.

The description of IB-MSS follows. **We implicitly assume that all algorithms after KGen and before Sign take mpk as input.**

Definition 7 (IB-MSS). *An IB-MSS scheme Π with message space $\mathcal{M}$ consists of a tuple of algorithms (MKGen, KGen, SKGen, RKGGen, OPKGen, OSKGen, Sign, Vrfy), described as follows:*

MKGen(1^λ): *On input the security parameter 1^λ , the randomized master key generator outputs a master public key mpk and a master secret key msk.*

KGen(1^λ): *On input the security parameter 1^λ , the randomized key generation algorithm outputs a public/private key pair (pk, sk).*

SKGen(msk, σ): *On input the master secret key msk and a sender identity $\sigma \in \{0, 1\}^*$, the deterministic sender key generator outputs a sender retrieval key srk.*

RKGGen(msk, ρ): *On input the master secret key msk and a receiver identity $\rho \in \{0, 1\}^*$, the deterministic receiver key generator outputs a receiver retrieval key rrk.*

OPKGen(srk, pk, ρ): *On input a sender retrieval key srk, the receiver's public key pk and a receiver identity $\rho \in \{0, 1\}^*$, the one-time deterministic public key generator outputs a one-time public key opk.*

OSKGen(opk, rrk, sk, σ): *On input a one-time public key opk, a receiver retrieval key rrk, the receiver's secret key sk and a sender identity $\sigma \in \{0, 1\}^*$, the one-time secret deterministic key generator outputs a one-time secret key osk, or $\perp$ indicating failure.*

Sign(osk, m): *On input a one-time secret key osk and a message $m \in \mathcal{M}$, the randomized signing algorithm outputs a signature τ .*

Vrfy(opk, m , τ): *On input a one-time public key opk, a message $m \in \mathcal{M}$, and a signature τ , the deterministic verification algorithm outputs 1 (valid) or 0 (invalid).*

Definition 8 (Correctness). *An Identity-Based Matchmaking Stealth Signature scheme is correct if, for every security parameter λ , every message $m \in \mathcal{M}$,*

and all identities $\sigma, \rho \in \{0, 1\}^*$,

$$\Pr \left[\text{Vrfy}(\text{opk}, m, \tau) = 1 \mid \begin{array}{l} (\text{mpk}, \text{msk}) \leftarrow_{\$} \text{MKGen}(1^\lambda); \\ (\text{pk}, \text{sk}) \leftarrow_{\$} \text{KGen}(1^\lambda); \\ \text{srk} := \text{SKGen}(\text{msk}, \sigma); \\ \text{rrk} := \text{RKGen}(\text{msk}, \rho); \\ \text{opk} := \text{OPKGen}(\text{srk}, \text{pk}, \rho); \\ \text{osk} := \text{OSKGen}(\text{opk}, \text{rrk}, \text{sk}, \sigma); \\ \tau \leftarrow_{\$} \text{Sign}(\text{osk}, m) \end{array} \right] = 1.$$

Definition 9 (Unforgeability). An Identity-Based Matchmaking Stealth Signature scheme Π satisfies existential unforgeability w.r.t. chosen message attacks if, for all valid PPT adversaries A , $\Pr[\text{EUFCMA}_A^\Pi(\lambda) = 1] \leq \text{negl}(\lambda)$.

Let $\mathcal{O} = \{\text{SKGen}(\text{msk}, \cdot), \text{RKGen}(\text{msk}, \cdot), \text{OSKGen}\mathcal{O}, \text{Sign}\mathcal{O}\}$. The adversary A in $\text{EUFCMA}_A^\Pi(\lambda)$ is valid if one of the following two conditions is satisfied:

- (Unforgeability w.r.t. the attributes) For all $(i, m) \in \mathcal{Q}_{\text{Sign}\mathcal{O}}$, let

$$(\cdot, \cdot, \cdot, \sigma', \rho') := \mathcal{Q}_{\text{OSKGen}\mathcal{O}}[i].$$

Then for all $\text{id} \in \mathcal{Q}_{\text{SKGen}(\text{msk}, \cdot)}$ and $\text{id}' \in \mathcal{Q}_{\text{RKGen}(\text{msk}, \cdot)}$,

$$\text{id} \notin \{\sigma', \rho'\} \wedge \text{id}' \notin \{\sigma', \rho'\}.$$

Additionally, whenever $(\cdot, \cdot, \text{true}, \sigma, \rho) \in \mathcal{Q}_{\text{OSKGen}\mathcal{O}}$, for all $\text{id} \in \mathcal{Q}_{\text{SKGen}(\text{msk}, \cdot)}$ and $\text{id}' \in \mathcal{Q}_{\text{RKGen}(\text{msk}, \cdot)}$ it holds that

$$\text{id} \notin \{\sigma, \rho\} \wedge \text{id}' \notin \{\sigma, \rho\}.$$

Moreover, whenever $(\cdot, \cdot, \cdot, \sigma, \rho) \in \mathcal{Q}_{\text{OSKGen}\mathcal{O}}$ we have

$$(\cdot, \cdot, \cdot, \rho, \sigma) \notin \mathcal{Q}_{\text{OSKGen}\mathcal{O}}.$$

- (Unforgeability w.r.t. the public key) We have $(\cdot, \cdot, \text{true}, \cdot, \cdot) \notin \mathcal{Q}_{\text{OSKGen}\mathcal{O}}$ and, letting

$$(\text{opk}^*, \text{osk}^*, \cdot, \sigma^*, \rho^*) := \mathcal{Q}_{\text{OSKGen}\mathcal{O}}[i^*],$$

for all $\sigma' \in \mathcal{Q}_{\text{SKGen}(\text{msk}, \cdot)}$ and $\rho' \in \mathcal{Q}_{\text{RKGen}(\text{msk}, \cdot)}$ it holds that

$$\sigma' \notin \{\sigma^*, \rho^*\} \wedge \rho' \notin \{\sigma^*, \rho^*\}.$$

The experiment $\text{EUFCMA}_A^\Pi(\lambda)$ is described in [Fig. 2](#).

$\text{EUFMA}_A^{\Pi}(\lambda)$	$\text{OSKGenO}(\text{opk}, \text{flag}, \sigma, \rho)$
$(\text{mpk}, \text{msk}) \leftarrow \text{MKGen}(1^\lambda)$	$\text{rrk} := \text{RKGen}(\text{msk}, \rho)$
$(\text{pk}, \text{sk}) \leftarrow \text{KGen}(1^\lambda)$	$\text{osk} := \text{OSKGen}(\text{opk}, \text{rrk}, \text{sk}, \sigma)$
$\mathcal{Q}_{\text{SignO}} := \emptyset$	$\mathcal{Q}_{\text{OSKGenO}} := \mathcal{Q}_{\text{OSKGenO}} \cup \{(\text{opk}, \text{osk}, \text{flag}, \sigma, \rho)\}$
$\mathcal{Q}_{\text{OSKGenO}} := \emptyset$	if $\text{flag} = \text{true}$ then return osk
$(i^*, m^*, \tau^*) \leftarrow \text{A}^{\mathcal{O}}(\text{mpk}, \text{pk})$	else return 1
if $i^* \notin [\mathcal{Q}_{\text{OSKGenO}}]$ then return 0	$\text{SignO}(i, m)$
$(\text{opk}^*, \text{osk}^*, \cdot, \sigma^*, \rho^*) :=$ $\mathcal{Q}_{\text{OSKGenO}}[i^*]$	if $i \notin [\mathcal{Q}_{\text{OSKGenO}}]$ then return $\perp$
$b_0 := \text{Vrfy}(\text{opk}^*, m^*, \tau^*) = 1$	$(\text{opk}, \text{osk}, \cdot, \cdot, \cdot) :=$ $\mathcal{Q}_{\text{OSKGenO}}[i]$
$b_1 := (i^*, m^*) \notin \mathcal{Q}_{\text{SignO}}$	if $\text{osk} = \perp$ then return $\perp$
$b_2 := (\text{opk}^*, \text{osk}^*, \text{true}, \sigma^*, \rho^*)$ $\notin \mathcal{Q}_{\text{OSKGenO}}$	$\tau \leftarrow \text{Sign}(\text{osk}, m)$
$b_3 := \text{osk}^* \neq \perp$	$\mathcal{Q}_{\text{SignO}} := \mathcal{Q}_{\text{SignO}} \cup \{(i, m)\}$
return $b_0 \wedge b_1 \wedge b_2 \wedge b_3$	return τ

Fig. 2. The IB-MSS unforgeability experiment $\text{EUFMA}_A^{\Pi}(\lambda)$. We implicitly assume that, other than the adversarial inputs, oracles have access to the internal state of the main experiment.

Definition 10 (Unlinkability). An Identity-Based Matchmaking Stealth Signature scheme Π satisfies unlinkability if, for all valid PPT adversaries $A = (A_0, A_1)$, $\Pr[\text{UNLINK}_A^{\Pi}(\lambda) = 1] \leq \frac{1}{2} + \text{negl}(\lambda)$.

Let $\mathcal{O} = \{\text{SKGen}(\text{msk}, \cdot), \text{RKGen}(\text{msk}, \cdot), \text{OSKGenO}, \text{SignO}\}$. The adversary A in $\text{UNLINK}_A^{\Pi}(\lambda)$ is valid if one of the following two conditions is satisfied:

- (Unlinkability w.r.t. attributes) For all $\sigma' \in \mathcal{Q}_{\text{SKGen}(\text{msk}, \cdot)}$ and $\rho' \in \mathcal{Q}_{\text{RKGen}(\text{msk}, \cdot)}$ we have

$$\sigma' \notin \{\sigma^*, \rho^*\} \wedge \rho' \notin \{\sigma^*, \rho^*\}.$$

Additionally,

$$(\cdot, \cdot, \cdot, \sigma^*, \rho^*) \notin \mathcal{Q}_{\text{OSKGenO}} \wedge (\cdot, \cdot, \cdot, \rho^*, \sigma^*) \notin \mathcal{Q}_{\text{OSKGenO}}.$$

Moreover, whenever $(\cdot, \cdot, \text{true}, \sigma, \rho) \in \mathcal{Q}_{\text{OSKGenO}}$, for all $\sigma' \in \mathcal{Q}_{\text{SKGen}(\text{msk}, \cdot)}$ and $\rho' \in \mathcal{Q}_{\text{RKGen}(\text{msk}, \cdot)}$ we have

$$\sigma' \notin \{\sigma, \rho\} \wedge \rho' \notin \{\sigma, \rho\}.$$

- (Unlinkability w.r.t. the public key) The above conditions hold, and additionally we have

$$(\cdot, \cdot, \text{true}, \cdot, \cdot) \notin \mathcal{Q}_{\text{OSKGenO}}.$$

The unlinkability experiment is described in [Fig. 3](#).

$\text{UNLINK}_A^H(\lambda)$	$\text{OSKGenO}(\text{opk}, \text{flag}, \sigma, \rho)$
$(\text{mpk}, \text{msk}) \leftarrow \text{MKGen}(1^\lambda)$	$\text{rrk} := \text{RKGen}(\text{msk}, \rho)$
$(\text{pk}, \text{sk}) \leftarrow \text{KGen}(1^\lambda)$	$\text{osk} := \text{OSKGen}(\text{opk}, \text{rrk}, \text{sk}, \sigma)$
$b \leftarrow \{0, 1\}$	$\mathcal{Q}_{\text{OSKGenO}} := \mathcal{Q}_{\text{OSKGenO}} \cup$
$\mathcal{Q}_{\text{OSKGenO}} := \emptyset$	$\{(\text{opk}, \text{osk}, \text{flag}, \sigma, \rho)\}$
$(\sigma^*, \rho^*, \text{st}) \leftarrow \text{A}_0^{\mathcal{O}}(\text{mpk}, \text{pk})$	if $\text{flag} = \text{true}$ then return osk
$(\text{pk}_0, \text{sk}_0) \leftarrow \text{KGen}(1^\lambda)$	else return $\perp$
$\text{srk} := \text{SKGen}(\text{msk}, \sigma^*)$	$\text{SignO}(i, m)$
$\text{rrk} := \text{RKGen}(\text{msk}, \rho^*)$	if $i \notin [\mathcal{Q}_{\text{OSKGenO}}]$ then return $\perp$
$\text{opk}_1 := \text{OPKGen}(\text{srk}, \text{pk}, \rho^*)$	$(\text{opk}, \text{osk}, \cdot, \cdot, \cdot) :=$
$\text{osk}_1 := \text{OSKGen}(\text{opk}_1, \text{rrk}, \text{sk}, \sigma^*)$	$\mathcal{Q}_{\text{OSKGenO}}[i]$
if $b = 0$ then $(\text{opk}_b, \text{osk}_b) := (\text{opk}_1, \text{osk}_1)$	if $\text{osk} = \perp$ then return $\perp$
else $(\text{opk}_b, \text{osk}_b) := (\text{pk}_0, \text{sk}_0)$	$\tau \leftarrow \text{Sign}(\text{osk}, m)$
$b' \leftarrow \text{A}_1^{\mathcal{O}}(\text{opk}_b, \text{osk}_b, \text{st})$	return τ
return $b \stackrel{?}{=} b'$	

Fig. 3. The IB-MSS unlinkability experiment $\text{UNLINK}_A^H(\lambda)$. We implicitly assume that, other than the adversarial inputs, oracles have access to the internal state of the main experiment.

5.2 IB-MSS from IB-NIKD

We now show how to instantiate IB-MSS from any IB-NIKD (Section 4.1), in combination with an EUF-CMA signature scheme (Section 4.2) where $\text{pk} = g^{\text{sk}}$, g is a generator of a group $\mathbb{G}$ of order q , and $\text{sk} \leftarrow \mathbb{Z}_q$. Notice that IB-NIKD is a natural candidate for constructing IB-MSS. Indeed, a one-time public key can be generated as $g^k \cdot \text{pk}$, where k is the shared IB-NIKD key derived from the sender and receiver identities, as already done in standard stealth addresses. This ties the resulting one-time secret key to sk , so the sender cannot spend it without recovering sk from pk . The receiver can then recover the corresponding one-time secret key and generate stealth signatures. Unlinkability reduces to the IND-security of the underlying IB-NIKD: replacing the challenge shared key with a uniform value makes $\text{osk} = k + \text{sk}$ uniform in $\mathbb{Z}_q$ and $\text{opk} = g^{\text{osk}}$ distributed identically to $\Pi_{\text{SIG}}.\text{KGen}(1^\lambda)$. Unforgeability is guaranteed by the EUF-CMA security of the signature scheme supporting public keys of type $\text{pk} = g^{\text{sk}}$. The formal construction follows.

Construction 1 (IB-MSS from IB-NIKD and signatures)

Let Π_{SIG} be an EUF-CMA signature scheme. Its message space is $\mathcal{M}$. Assume that $\text{KGen}(1^\lambda)$ outputs $(\text{pk} = g^{\text{sk}}, \text{sk} \leftarrow \mathbb{Z}_q)$, where $\mathbb{G}$ is a group of order q . Let Π_{NIKD} be an Identity-Based Non-Interactive Key Distribution scheme with key space $\mathcal{SHK} = \mathbb{Z}_q$. We construct our IB-MSS scheme Π_{IBMSS} as follows:

$\text{MKGen}(1^\lambda)$: Output $(\text{mpk}, \text{msk}) \leftarrow \text{II}_{\text{NIKD}}.\text{Setup}(1^\lambda)$.
 $\text{KGen}(1^\lambda)$: Output $(\text{pk}(=g^{\text{sk}}), \text{sk}) \leftarrow \text{II}_{\text{SIG}}.\text{KGen}(1^\lambda)$.
 $\text{SKGen}(\text{msk}, \text{id})$ and $\text{RKGen}(\text{msk}, \text{id})$: Output $\text{II}_{\text{NIKD}}.\text{Extract}(\text{mpk}, \text{msk}, \text{id})$.
 $\text{OPKGen}(\text{srk}, \text{pk}, \rho)$: Set $k = \text{II}_{\text{NIKD}}.\text{SharedKey}(\text{mpk}, \text{srk}, \rho)$ and output $g^k \cdot \text{pk}$.
 $\text{OSKGen}(\text{opk}, \text{rrk}, \text{sk}, \sigma)$: Set $k = \text{II}_{\text{NIKD}}.\text{SharedKey}(\text{mpk}, \text{rrk}, \sigma)$. If $g^{k+\text{sk}} = \text{opk}$,
output $k + \text{sk}$, else output $\perp$.
 $\text{Sign}(\text{osk}, m)$: Output $\text{II}_{\text{SIG}}.\text{Sign}(\text{osk}, m)$.
 $\text{Vrfy}(\text{opk}, m, \tau)$: Output $\text{II}_{\text{SIG}}.\text{Vrfy}(\text{opk}, m, \tau)$.

In the following theorem, we state that II_{IBMSS} satisfies the security properties we require.

Theorem 2. *Assuming that II_{NIKD} is IND-secure and II_{SIG} is EUF-CMA secure, the IB-MSS scheme II_{IBMSS} described above satisfies correctness, unforgeability, and unlinkability.*

Proof. We will prove that the IB-MSS scheme II_{IBMSS} satisfies correctness, unlinkability, and unforgeability under the assumptions that II_{NIKD} is IND-secure and II_{SIG} is EUF-CMA secure.

Correctness. By the correctness of II_{NIKD} , for any sender identity σ and receiver identity ρ , the shared keys computed by both parties are equal:

$$k = \text{II}_{\text{NIKD}}.\text{SharedKey}(\text{mpk}, \text{srk}, \rho) = \text{II}_{\text{NIKD}}.\text{SharedKey}(\text{mpk}, \text{rrk}, \sigma),$$

where $\text{srk} = \text{II}_{\text{NIKD}}.\text{Extract}(\text{mpk}, \text{msk}, \sigma)$ and $\text{rrk} = \text{II}_{\text{NIKD}}.\text{Extract}(\text{mpk}, \text{msk}, \rho)$.

The one-time public and secret keys are then:

$$\text{opk} = g^k \cdot \text{pk}, \quad \text{osk} = k + \text{sk}.$$

Since (opk, osk) is a valid key pair in II_{SIG} , and II_{SIG} is correct, any signature $\tau = \text{II}_{\text{SIG}}.\text{Sign}(\text{osk}, m)$ will verify:

$$\text{II}_{\text{SIG}}.\text{Vrfy}(\text{opk}, m, \tau) = 1,$$

for all messages $m \in \mathcal{M}$. Thus, the IB-MSS scheme is correct.

Unlinkability. Assume, for contradiction, that there exists a PPT adversary $\mathbf{A}$ that can distinguish between a key pair generated by the IB-MSS scheme and a random key pair with non-negligible advantage ϵ . We subdivide the proof into two cases: when the adversary $\mathbf{A}$ is valid for unlinkability w.r.t. the attributes, and when $\mathbf{A}$ is valid for unlinkability w.r.t. the public key.

Unlinkability w.r.t. the attributes We construct an adversary $\mathbf{A}_{\text{NIKD}}$ that breaks the IND-security of II_{NIKD} by simulating the unlinkability experiment for $\mathbf{A} = (\mathbf{A}_0, \mathbf{A}_1)$.

1. Setup:

- $\mathbf{A}_{\text{NIKD}}$ receives mpk from the IND-security challenger.
- Generate $(\text{pk}, \text{sk}) \leftarrow \text{KGen}(1^\lambda)$.

2. Oracle Simulation:

- **Key Generation (SKGen, RKGen):** Use the extraction oracle to obtain and return keys.
 - **One-Time Key Generation (OSKGenO):**
 - Upon receiving a tuple $(\text{opk}, \text{flag}, \sigma, \rho)$ from A_0 or A_1 , query the RevealO oracle with input (ρ, σ) to receive k . If $\text{opk} \neq g^k \cdot \text{pk}$, set $\text{osk} = \perp$, else set $\text{osk} := k + \text{sk}$.
 - If $\text{flag} = \text{true}$, return osk ; else, return 1.
 - **Signing (SignO):** Upon receiving (i, m) , return $\perp$ if i is out of range; otherwise retrieve $(\cdot, \text{osk}, \cdot, \cdot, \cdot) := \mathcal{Q}_{\text{OSKGenO}}[i]$ and return $\perp$ if $\text{osk} = \perp$; else return $\Pi_{\text{SIG}}.\text{Sign}(\text{osk}, m)$.
3. **Challenge Phase:**
- Run $A_0^{\mathcal{O}}(\text{mpk}, \text{pk})$ to obtain $(\sigma^*, \rho^*, \text{st})$.
 - Send $(\sigma^*, \rho^*, \text{st})$ to the IND-security challenger and receive the challenge key k^* (either real or random).
 - Set $\text{osk}^* := k^* + \text{sk}$ and $\text{opk}^* := g^{k^*} \cdot \text{pk}$.
 - Run $A_1^{\mathcal{O}}(\text{opk}^*, \text{osk}^*, \text{st})$ to obtain a guess b' .
4. **Finalization:**
- Output b' as A_{NIKD} 's guess in the IND-security experiment.

Since A is valid for unlinkability w.r.t. attributes, A_0 never queries SKGen or RKGen on σ^* or ρ^* ; hence A_{NIKD} never queries Extract on σ^* or ρ^* . Moreover, A never makes an OSKGenO query on (σ^*, ρ^*) or on (ρ^*, σ^*) , so A_{NIKD} never queries RevealO on (σ^*, ρ^*) or on (ρ^*, σ^*) . Therefore, A_{NIKD} is valid for the IND-security experiment. If k^* is real, then $(\text{opk}^*, \text{osk}^*)$ is distributed as an IB-MSS-derived key pair; if k^* is uniform, then osk^* is uniform in $\mathbb{Z}_q$ and $(\text{opk}^*, \text{osk}^*)$ is distributed as $\Pi_{\text{SIG}}.\text{KGen}(1^\lambda)$. Thus, any non-negligible unlinkability advantage implies a non-negligible IND advantage, contradicting the IND-security of Π_{NIKD} .

Unlinkability w.r.t the public key. The same reduction applies. In the IND-random case, we have $k^* \leftarrow \mathbb{Z}_q$ independent of $\text{sk} \leftarrow \mathbb{Z}_q$, hence $\text{osk}^* = k^* + \text{sk}$ is uniform in $\mathbb{Z}_q$ and $\text{opk}^* = g^{k^*} \cdot \text{pk} = g^{\text{osk}^*}$. Therefore $(\text{opk}^*, \text{osk}^*)$ is distributed identically to a fresh key pair sampled from $\Pi_{\text{SIG}}.\text{KGen}(1^\lambda)$ (i.e., the $b = 1$ branch of the unlinkability experiment). Thus any non-negligible distinguishing advantage would contradict the IND-security of Π_{NIKD} .

Unforgeability.

Let A be a valid PPT adversary in the experiment $\text{EUF-CMA}_A^{\Pi_{\text{IBMSS}}}(\lambda)$. Let (i^*, m^*, τ^*) be its output, and let

$$(\text{opk}^*, \text{osk}^*, \text{flag}^*, \sigma^*, \rho^*) := \mathcal{Q}_{\text{OSKGenO}}[i^*].$$

If the experiment returns 1, then $\text{osk}^* \neq \perp$ and $\text{Vrfy}(\text{opk}^*, m^*, \tau^*) = 1$ while $(i^*, m^*) \notin \mathcal{Q}_{\text{SignO}}$ and $(\text{opk}^*, \text{osk}^*, \text{true}, \sigma^*, \rho^*) \notin \mathcal{Q}_{\text{OSKGenO}}$. We show that this contradicts the EUF-CMA security of Π_{SIG} .

Since $\text{osk}^* \neq \perp$, in Π_{IBMSS} we have $\text{opk}^* = g^{\text{osk}^*}$; thus $(\text{opk}^*, \text{osk}^*)$ is a valid key pair for Π_{SIG} , and the oracle $\text{SignO}(i^*, \cdot)$ is exactly a signing oracle for Π_{SIG} under secret key osk^* (and public key opk^*).

Moreover, under Construction 1 we assume $\Pi_{\text{SIG}}.\text{KGen}(1^\lambda)$ outputs $(\text{pk} = g^{\text{sk}}, \text{sk} \leftarrow \mathbb{Z}_q)$ and MKGen is run independently of KGen . Therefore, whenever

$\text{osk}^* \neq \perp$ we can write $\text{osk}^* = k^* + \text{sk}$ for some $k^* \in \mathbb{Z}_q$ independent of sk , implying that osk^* is uniform in $\mathbb{Z}_q$ and thus $\text{opk}^* = g^{\text{osk}^*}$ is distributed exactly as a public key output by $\Pi_{\text{SIG}}.\text{KGen}(1^\lambda)$.

We now distinguish the two validity cases.

Unforgeability w.r.t. the attributes. By validity, whenever $\mathcal{A}$ receives a leaked one-time secret key via a $\text{flag} = \text{true}$ query, $\mathcal{A}$ does not also obtain a retrieval key for either identity in that query; hence $\mathcal{A}$ cannot recover the long-term signing key sk from any leaked one-time secret key. In addition, the winning condition enforces that $\mathcal{A}$ does not obtain osk^* via a $\text{flag} = \text{true}$ query. Therefore, relative to the forgery index i^* , the only way $\mathcal{A}$ obtains signatures under opk^* is through oracle calls $\text{SignO}(i^*, \cdot)$, which return $\Pi_{\text{SIG}}.\text{Sign}(\text{osk}^*, \cdot)$. Because $(i^*, m^*) \notin Q_{\text{SignO}}$, the pair (m^*, τ^*) constitutes an EUF-CMA forgery for Π_{SIG} under public key opk^* .
Unforgeability w.r.t. the public key. In this case, $\mathcal{A}$ never makes any OSKGenO query with $\text{flag} = \text{true}$, so it never learns any one-time secret key at all; the same EUF-CMA argument applies directly to the forged index i^* .

Therefore, $\Pr[\text{EUF-CMA}_{\mathcal{A}}^{\Pi_{\text{IBMSS}}}(\lambda) = 1]$ is negligible, as claimed.

5.3 Certification Authority

In our protocol, a Certification Authority (CA) ensures that participants comply with specific regulations, such as AML/KYC requirements, while preserving privacy. The CA facilitates secure generation of shared secrets between parties based on certified attributes, enabling both privacy and regulatory compliance. We examine two CA models: centralized and decentralized, both constructed using our IB-MSS scheme instantiated from any IB-NIKD Π_{NIKD} (Section 5.2).

Centralized CA A centralized CA, such as a major exchange (*e.g.*, Coinbase or Kraken), performs Know Your Customer (KYC) procedures to verify user identities for compliance. The CA issues certificates that confirm specific attributes, such as user identity or location, after users complete KYC.

Consider the case where Alice and Bob, both users of a centralized exchange C , have completed KYC and now wish to transact. Alice must satisfy attribute σ and Bob must satisfy attribute ρ . The CA C operates as follows:

- C holds a master secret key msk and publishes a master public key mpk .
- Alice requests a private key (certificate) for her attribute σ , and C computes $D_A := \Pi_{\text{NIKD}}.\text{Extract}(\text{mpk}, \text{msk}, \sigma)$, providing it to Alice.
- Bob similarly requests his certificate $D_B := \Pi_{\text{NIKD}}.\text{Extract}(\text{mpk}, \text{msk}, \rho)$ from C .

Decentralized CA In a decentralized setup, protocols like VetKeys [9], developed by DFINITY, enable secure key derivation and policy compliance without relying on centralized authorities. VetKeys supports on-chain key derivation and verification, allowing users to comply with policies through decentralized applications (DApps). The process, depicted in Fig. 4, proceeds as follows:

- Alice generates a transport key pair $(\text{tpk}_A, \text{tsk}_A)$ and submits her transport public key tpk_A along with her policy attribute σ to a DApp D .

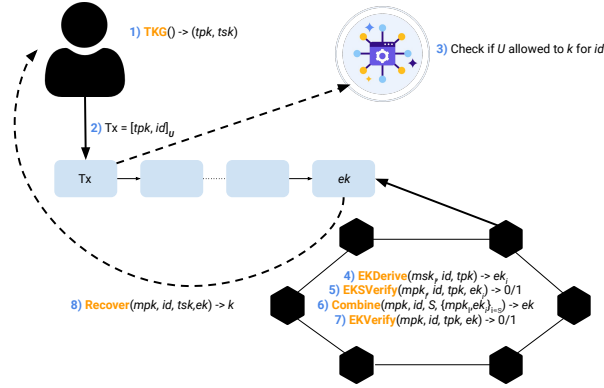


Fig. 4. VetKeys scheme in the blockchain context.

- The DApp verifies Alice’s compliance with the policy σ (*e.g.*, using zero-knowledge proofs or on-chain credentials).
- If compliant, the DApp nodes execute the VetKeys protocol to generate an encrypted derived key ek_A , which is stored on the blockchain.
- Alice retrieves ek_A and, using her transport secret key tsk_A , recovers her derived key $D_A = \Pi_{\text{NIKD}}.\text{Extract}(mpk, msk, \sigma)$.
- Bob similarly obtains his derived key $D_B = \Pi_{\text{NIKD}}.\text{Extract}(mpk, msk, \rho)$.

Shared Secret Computation In both centralized and decentralized CA models, Alice and Bob use their derived keys to compute a shared secret. Alice computes

$$k_A := \Pi_{\text{NIKD}}.\text{SharedKey}(mpk, D_A, \rho),$$

and Bob computes

$$k_B := \Pi_{\text{NIKD}}.\text{SharedKey}(mpk, D_B, \sigma).$$

By correctness, $k_A = k_B$; we denote the common value by k . This is the IB-NIKD key used by our IB-MSS construction to derive the one-time keys, ensuring compliance and privacy in both models.

6 Implementation and Experiments

We aim to illustrate the practicality of our proposed approach within the Bitcoin ecosystem, highlighting its capability to achieve transaction accountability while preserving user privacy. To this end, we first describe our instantiation and then present a comprehensive performance evaluation of our implementation, which is based on instantiating our IB-MSS (Section 5.2) from the Sakai-Ohgishi-Kasahara [27] IB-NIKD and Schnorr signatures [31]. All pairing-based computations are performed off-chain; the Bitcoin-side key-tweak that turns the shared secret into a one-time `secp256k1` output key is described in Section 7. In Appendix A, we provide a detailed analysis of parsing time costs in Bitcoin.

6.1 IB-MSS Instantiation

Let $\mathbb{G}_1$ and $\mathbb{G}_2$ be groups of prime order p and let $e : \mathbb{G}_1 \times \mathbb{G}_1 \rightarrow \mathbb{G}_2$ be a bilinear pairing.

Model the hash functions $H_1 : \{0,1\}^* \rightarrow \mathbb{G}_1$ and $H_2 : \mathbb{G}_2 \rightarrow \mathbb{Z}_p$, and $H_3 : \mathbb{G}_1 \times \{0,1\}^* \rightarrow \mathbb{Z}_p$ as random oracles. The message space is $\{0,1\}^*$.

MKGen(1^λ): Sample $\text{msk} \leftarrow \mathbb{Z}_p^*$, choose generators $g_1 \in \mathbb{G}_1$, $g_2 \in \mathbb{G}_2$, and set $\text{mpk} = (\mathbb{G}_1, \mathbb{G}_2, p, g_1, g_2, g_1^{\text{msk}})$. Output (mpk, msk) .

KGen(1^λ): Sample $\text{sk} \leftarrow \mathbb{Z}_p$ and output $(\text{pk} = g_1^{\text{sk}}, \text{sk})$.

SKGen(msk, id) and **RKGen**(msk, id): Return $H_1(\text{id})^{\text{msk}}$.

OPKGen($\text{srk}, \text{pk}, \rho$): Compute $k := H_2(e(\text{srk}, H_1(\rho)))$ and output $\text{opk} := g_1^k \text{pk}$.

OSKGen($\text{opk}, \text{rrk}, \text{sk}, \sigma$): Compute $k := H_2(e(H_1(\sigma), \text{rrk}))$. If $g_1^{k+\text{sk}} = \text{opk}$ output $\text{osk} := k + \text{sk}$, else output $\perp$.

Sign(osk, m): Sample $r \leftarrow \mathbb{Z}_p$, let $R := g_1^r$ and $c := H_3(R, m)$, set $s := r - c \text{osk} \bmod p$, and output $\tau := (R, s)$.

Vrfy(opk, m, τ): Parse $\tau = (R, s)$, compute $c := H_3(R, m)$, and accept iff $R = g_1^s \text{opk}^c$.

6.2 Performance Evaluation

The proof-of-concept is implemented using Python 3.10.10, leveraging Charm 0.50 [3]. Our pairing-based scheme is instantiated using the SS512 curve. The experiments were conducted on a machine equipped with an Intel Core i7-12700 CPU (2.10GHz) and 32GB of RAM, running Manjaro Linux 22.1.0. The source code for our implementation is available on GitHub [20].

Operation	Minimum (ms)	Average (ms)	Maximum (ms)
MKGen	1.228	1.250	1.298
SKGen & RKGen	1.942	1.996	2.139
OPKGen	2.340	2.497	2.656
OSKGen	2.326	2.445	2.648

Table 1. Time costs in milliseconds for computing cryptographic functions.

Table 1 shows the time costs (in milliseconds) of the primary cryptographic operations. We conducted 100 independent runs for each operation, repeating each experiment ten times to extract minimum, average, and maximum execution times. For the main cryptographic operations, **OPKGen** and **OSKGen**, the average execution times were 2.497 ms and 2.445 ms, respectively.

Compared with a standard P2WPKH transaction or with the DKSAP protocol, the primary overhead of our scheme manifests during the off-chain address derivation and key retrieval phases. We analyze the costs in two categories: setup operations and transactional operations.

The system initialization algorithms (MKGen, SKGen, RKGen) are specific to our protocol but are executed infrequently: MKGen runs only once during system genesis, and key generation occurs only once per user upon registration. Consequently, their amortized computational cost per transaction is negligible.

For the transactional operations (OPKGen and OSKGen), the overhead depends on the comparison baseline. When compared against standard P2WPKH, a standard sender performs no cryptographic derivation for the address (resulting in a cost of approximately 0 ms). For example, the overhead of our scheme is the full execution time of OPKGen, which averages 2.497 ms. Conversely, when compared to DKSAP, the sender in that protocol performs elliptic curve scalar multiplications at an average of 0.894 ms. Relative to this baseline, the OPKGen operation introduces an additional overhead of approximately 1.6 ms.

Despite this increase, the absolute cost remains highly practical. An overhead of roughly 1.6 ms represents a trivial wait time for a human user, especially when compared with the network latency of propagating a transaction (which takes seconds) or the block mining time (which takes a few minutes).

7 Practical Considerations

To bridge the gap between our theoretical model and real-world deployment, we present a concrete workflow demonstrating how a Bitcoin user (Alice) transfers funds to a recipient (Bob) using our IB-MSS scheme. This implementation relies on standard Bitcoin RPC commands and derives all public salting data from standard transaction fields, without requiring additional OP_RETURN outputs or custom witness data.

7.1 System Setup

Let pk_A and pk_B be the public keys of Alice and Bob, respectively, on the curve `secp256k1` with generator G and prime order n . In this workflow, we take id_A and id_B to be encodings of pk_A and pk_B , respectively, so that the receiver can recover id_A from standard witness data; when a transaction has multiple inputs, we let pk_A be the public key revealed for the first input. In the attribute-driven setting, id can instead denote the certified attribute string(s) issued by the CA, and the same workflow applies. Separately, let $e : \mathbb{G}_1 \times \mathbb{G}_1 \rightarrow \mathbb{G}_2$ be a bilinear pairing for groups $\mathbb{G}_1$ and $\mathbb{G}_2$ of prime order p , with associated random oracles $H_1 : \{0, 1\}^* \rightarrow \mathbb{G}_1$ and $H_2 : \mathbb{G}_2 \rightarrow \mathbb{Z}_p$. Let $\chi : \{0, 1\}^* \rightarrow \mathbb{Z}_n$ be a hash-to-scalar map (interpreting its input as an integer modulo n). All pairing computations are performed off-chain; the blockchain carries only `secp256k1` public keys and standard transaction data. From now on, we use additive notation when dealing with `secp256k1` points.

Before any transactions can occur, the CA performs a one-time global setup phase.

1. **Global Setup:** The CA runs $\text{MKGen}(1^\lambda)$ to generate the system's master public key (mpk) and master secret key (msk).

2. **User Registration:** Alice and Bob authenticate themselves to the CA once to obtain their retrieval keys.
 - **Alice (Sender)** Let id_A be Alice’s identity. The CA sends $\text{srk}_A := \text{SKGen}(\text{msk}, \text{id}_A) = H_1(\text{id}_A)^{\text{msk}}$ to Alice.
 - **Bob (Recipient)** Let id_B be Bob’s identity. The CA sends $\text{rrk}_B := \text{RKGen}(\text{msk}, \text{id}_B) = H_1(\text{id}_B)^{\text{msk}}$ to Bob.

7.2 Transaction Workflow Example

Consider a scenario where **Alice** (Sender) with identity id_A and sender retrieval key srk_A intends to pay **Bob** (Recipient) with identity id_B . The workflow proceeds as follows.

Step 1: Address Derivation (Off-Chain). Unlike traditional stealth addresses that explicitly publish an ephemeral point, our implementation derives a public salt R from standard transaction data. At the abstract level (Construction 1), the sender computes $\text{opk} = g^k \cdot \text{pk}$ and the receiver derives $\text{osk} = k + \text{sk}$ in a prime-order group. In Bitcoin, we instead map the pairing-derived shared secret $k \in \mathbb{Z}_p$ into a `secp256k1` scalar offset $\delta \in \mathbb{Z}_n$ via a KDF salted by a public value R taken from the transaction, and set $\text{pk}_Q = \text{pk}_B + \delta G$ (and $\text{osk}_Q = \text{sk}_B + \delta$). The following steps describe the practical Bitcoin-side implementation of $\text{OPKGen}(\text{srk}_A, \text{pk}_B, \text{id}_B)$, together with a salting step to ensure one-time addresses.

1. **Shared key derivation:** Alice uses her sender retrieval key srk_A and Bob’s identity id_B to compute the shared key:

$$k := H_2(e(\text{srk}_A, H_1(\text{id}_B)))$$

2. **Public witness value (R):** In the spirit of silent payments [14], Alice derives R deterministically from public keys that will appear on-chain when spending her UTXOs. Concretely, let $\text{pk}_{A,1}, \dots, \text{pk}_{A,m}$ be the `secp256k1` public keys revealed in the witness stacks of the m inputs (*e.g.*, in P2WPKH, each input witness includes the spending public key). Alice sets

$$R := \text{SHA256}(\text{pk}_{A,1} \parallel \dots \parallel \text{pk}_{A,m}),$$

where the concatenation uses the standard serialized encodings and the inputs are ordered as they appear in the transaction. This value is public and available to Bob when scanning the chain. If Alice reuses the same set of input public keys for multiple payments to Bob, then R repeats; in typical wallet practice, keys are rotated so this value changes across transactions.

3. **Stealth Address Generation:** First, Alice computes the digest by hashing the concatenation of the shared secret k and the public salt R :

$$\text{digest} := \text{SHA256}(k \parallel \text{SHA256}(R)).$$

All hashes are evaluated over canonical byte encodings: public keys use the standard serialized encodings, and k is encoded as a fixed-length integer modulo p . Let $\delta := \chi(\text{digest}) \in \mathbb{Z}_n$. Then, to ensure that *only* Bob can spend the funds, she combines the derived offset δ with Bob's static public key pk_B to obtain the public key pk_Q of the new stealth address Q :

$$\text{pk}_Q := \text{pk}_B + \delta G$$

Since Alice does not know Bob's private key corresponding to pk_B , she can compute Bob's stealth address pk_Q but cannot derive the private key required to spend from it. Finally, she computes the stealth address Q corresponding to pk_Q using the P2WPKH Bitcoin function:

$$Q := \text{P2WPKH.PublicKeyToAddress}(\text{pk}_Q)$$

Step 2: The Transaction (On-Chain). Alice broadcasts the transaction (Tx_{send}) to the Bitcoin network.

- **Input:** Alice's UTXO(s). For a standard P2WPKH spend, each input witness includes the corresponding public key; Bob uses these to compute R .
- **Output:** A standard P2WPKH output paying to the derived address Q .

Since R is derived from standard transaction data, no additional `OP_RETURN` output or custom witness data is needed. This makes the transaction structure identical to a standard Pay-to-Witness-Public-Key-Hash (P2WPKH) transaction, maximizing privacy.

Step 3: Detection and Spending. Bob monitors the blockchain for incoming transactions. For a candidate transaction Tx , he runs the following procedure to recover the one-time secret key (a Bitcoin adaptation of $\text{OSKGen}(\text{pk}_Q, \text{rrk}_B, \text{sk}_B, \text{id}_A)$):

1. **Sender identity and salt:** Bob extracts pk_A from the witness data of the first input and sets id_A accordingly. He also extracts the input public keys $\text{pk}_{A,1}, \dots, \text{pk}_{A,m}$ and computes R as in Step 1.
2. **Shared key retrieval:** Bob uses his receiver retrieval key rrk_B and the sender identity id_A to retrieve the shared key k :

$$k := H_2(e(H_1(\text{id}_A), \text{rrk}_B))$$

3. **Re-derivation and spending:** Bob recomputes the digest as in Step 1, sets $\delta := \chi(\text{digest})$, and recomputes the one-time public key $\text{pk}_Q := \text{pk}_B + \delta G$. He checks if the output address matches the address corresponding to pk_Q . If so, Bob computes the one-time secret key $\text{osk}_Q := \text{sk}_B + \delta$ and can spend the stealth output. In particular, since Bob is the only entity possessing sk_B , he is the only party capable of generating a valid signature for pk_Q .

To further demonstrate the feasibility of our protocol, we executed two accountable stealth transactions on the Bitcoin Testnet. These transactions were performed in both centralized [1] and decentralized [2] settings, using a modified version of the VetKeys protocol for the decentralized scenario.

Our protocol ensures that no additional information is embedded in transactions, making them indistinguishable from standard Bitcoin transactions. As a result, the transaction fees remain equivalent to those of standard Pay-to-Witness-Public-Key-Hash (P2WPKH) transactions. P2WPKH, introduced with the Segregated Witness (SegWit) update through BIP141 [16], was chosen over Pay-to-Public-Key-Hash (P2PKH) due to its lower transaction fees [11].

8 Conclusions and Open Problems

This paper introduced “stealth addresses with attributes,” an innovative extension of stealth address protocols that enhances accountability while preserving privacy in Bitcoin transactions. By embedding compliance attributes into the stealth address framework, we achieve both sender and recipient accountability without compromising anonymity. The integration of Identity-Based Matchmaking Signatures (IB-MSS) further ensures verifiable compliance within Bitcoin’s existing infrastructure.

However, two main challenges remain. The first is the computational overhead of parsing and verifying transactions with embedded attributes, which impacts scalability as transaction volumes grow. Future work should focus on optimizing parsing and verification processes to handle larger loads efficiently. The second challenge involves implementing practical and scalable zero-knowledge proofs for recipient accountability within the IB-MSS framework. Addressing this is essential to ensuring the protocol remains efficient and secure in real-world applications.

References

1. Blockstream explorer (2023), <https://blockstream.info/testnet/tx/e65a99f0a260300c010270c35636d0497c2a89eb07bc04aa148fb71b076f1408>
2. Blockstream explorer (2023), <https://blockstream.info/testnet/tx/b5e74da90c37fdgbaafc9afa6a845147cbff53ae9818d1426ffb27b16ab936f4>
3. Akinyele, J.A., Garman, C., Miers, I., Pagano, M.W., Rushanan, M., Green, M., Rubin, A.D.: Charm: a framework for rapidly prototyping cryptosystems. *Journal of Cryptographic Engineering* **3**, 111–128 (2013), <https://api.semanticscholar.org/CorpusID:2876079>
4. Apple: Secure Enclave (2025), <https://support.apple.com/guide/security/secure-enclave-sec59b0b31ff/web>
5. Ateniese, G., Francati, D., Nuñez, D., Venturi, D.: Match me if you can: Matchmaking encryption and its applications. In: Boldyreva, A., Micciancio, D. (eds.) *CRYPTO 2019, Part II*. LNCS, vol. 11693, pp. 701–731. Springer, Cham (Aug 2019). https://doi.org/10.1007/978-3-030-26951-7_24

6. Backes, M., Hanzlik, L., Klucznik, K., Schneider, J.: Signatures with flexible public key: Introducing equivalence classes for public keys. In: Peyrin, T., Galbraith, S. (eds.) ASIACRYPT 2018, Part II. LNCS, vol. 11273, pp. 405–434. Springer, Cham (Dec 2018). https://doi.org/10.1007/978-3-030-03329-3_14
7. Boneh, D., Franklin, M.K.: Identity-based encryption from the Weil pairing. In: Kilian, J. (ed.) CRYPTO 2001. LNCS, vol. 2139, pp. 213–229. Springer, Berlin, Heidelberg (Aug 2001). https://doi.org/10.1007/3-540-44647-8_13
8. Buterin, V., Illum, J., Nadler, M., Schär, F., Soleimani, A.: Blockchain privacy and regulatory compliance: Towards a practical equilibrium. Available at SSRN (2023)
9. Cerulli, A., Connolly, A., Neven, G., Preiss, F.S., Shoup, V.: vetkeys: How a blockchain can keep many secrets. Cryptology ePrint Archive (2023)
10. Fauzi, P., Meiklejohn, S., Mercer, R., Orlandi, C.: Quisquis: A new design for anonymous cryptocurrencies. In: Galbraith, S.D., Moriai, S. (eds.) ASIACRYPT 2019, Part I. LNCS, vol. 11921, pp. 649–678. Springer, Cham (Dec 2019). https://doi.org/10.1007/978-3-030-34578-5_23
11. FixedFloat: Bitcoin address formats and performance comparison (2022), <https://fixedfloat.com/en/blog/guides/bitcoin-address-formats>
12. Fleischhacker, N., Krupp, J., Malavolta, G., Schneider, J., Schröder, D., Simkin, M.: Efficient unlinkable sanitizable signatures from signatures with re-randomizable keys. In: Cheng, C.M., Chung, K.M., Persiano, G., Yang, B.Y. (eds.) PKC 2016, Part I. LNCS, vol. 9614, pp. 301–330. Springer, Berlin, Heidelberg (Mar 2016). https://doi.org/10.1007/978-3-662-49384-7_12
13. Forum, B.: Bytecoin (2011), <https://bitcointalk.org/index.php?topic=199.msg1670#msg1670>
14. josibake: Silent payments (2023), <https://github.com/bitcoin/bips/blob/master/bip-0352.mediawiki>
15. Liu, Z., Yang, G., Wong, D.S., Nguyen, K., Wang, H.: Key-insulated and privacy-preserving signature scheme with publicly derived public key. In: IEEE EuroS&P. pp. 215–230. IEEE (2019)
16. Lombrozo, E.: Segregated witness (consensus layer) (2015), <https://github.com/bitcoin/bips/blob/master/bip-0141.mediawiki#p2wpkh>
17. Lyastani, S.G., Schilling, M., Neumayr, M., Backes, M., Bugiel, S.: Is fido2 the kingslayer of user authentication? a comparative usability study of fido2 passwordless authentication. In: 2020 IEEE Symposium on Security and Privacy (SP). pp. 268–285. IEEE (2020)
18. Meiklejohn, S., Mercer, R.: Möbius: Trustless tumbling for transaction privacy. PoPETs **2018**(2), 105–121 (Apr 2018). <https://doi.org/10.1515/popets-2018-0015>
19. Monero: Moneropedia. <https://www.getmonero.org/resources/moneropedia/stealthaddress.html> (2025)
20. Mongardini, A.M., Friolo, D., Ateniese, G.: Accountable stealth transactions with attributes - github repository. <https://github.com/Alb4tro/Attribute-Driven-Accountability-in-Bitcoin-Transactions> (2026)
21. Paterson, K.G., Srinivasan, S.: On the relations between non-interactive key distribution, identity-based encryption and trapdoor discrete log groups. Des. Codes Cryptogr. **52**(2), 219–241 (2009). <https://doi.org/10.1007/S10623-009-9278-Y>, <https://doi.org/10.1007/s10623-009-9278-y>

22. Perez, R., Sailer, R., van Doorn, L., et al.: vtpm: virtualizing the trusted platform module. In: Proc. 15th Conf. on USENIX Security Symposium. pp. 305–320 (2006)
23. Privacy, S.F.: View tags: How one byte will reduce monero wallet sync times by 40<https://localmonero.co/knowledge/view-tags-reduce-monero-sync-time>
24. Pu, S., Thyagarajan, S.A.K., Döttling, N., Hanzlik, L.: Post quantum fuzzy stealth signatures and applications. In: Meng, W., Jensen, C.D., Cremers, C., Kirda, E. (eds.) Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security, CCS 2023, Copenhagen, Denmark, November 26–30, 2023. pp. 371–385. ACM (2023).
<https://doi.org/10.1145/3576915.3623148>,
<https://doi.org/10.1145/3576915.3623148>
25. Rivest, R.L., Shamir, A., Wagner, D.A.: Time-lock puzzles and timed-release crypto (1996), <https://people.csail.mit.edu/rivest/pubs/RSW96.pdf>
26. van Saberhagen, N.: Cryptonote v 2.0 (2013),
<https://cryptonote.org/whitepaper.pdf>
27. Sakai, R., Ohgishi, K., Kasahara, M.: Cryptosystems based on pairing. The 2000 Symposium on Cryptography and Information Security pp. 26–28 (2000)
28. Salvo, M.D.: Tornado cash user ‘dusts’ hundreds of public wallets—including celebs jimmy fallon, steve aoki and logan paul (2022),
<https://decrypt.co/107090/tornado-cash-dusts-public-wallets-jimmy-fallon-brian-armstrong-steve-aoki-logan-paul?amp=1>
29. Samsung: Knox Vault (2025),
<https://docs.samsungknox.com/admin/fundamentals/whitepaper/samsung-knox-mobile-security/system-security/knox-vault/>
30. Sasson, E.B., Chiesa, A., Garman, C., Green, M., Miers, I., Tromer, E., Virza, M.: Zerocash: Decentralized anonymous payments from bitcoin. In: 2014 IEEE symposium on security and privacy. pp. 459–474. IEEE (2014)
31. Schnorr, C.: Efficient identification and signatures for smart cards. In: Brassard, G. (ed.) Advances in Cryptology - CRYPTO ’89, 9th Annual International Cryptology Conference, Santa Barbara, California, USA, August 20–24, 1989, Proceedings. Lecture Notes in Computer Science, vol. 435, pp. 239–252. Springer (1989). https://doi.org/10.1007/0-387-34805-0_22,
https://doi.org/10.1007/0-387-34805-0_22
32. Todd, P.R.: Stealth addresses in bitcoin.,
<https://github.com/bitcoin/bips/blob/master/bip-0032.mediawiki>
33. TREASURY, U.D.O.T.: U.s. treasury sanctions notorious virtual currency mixer tornado cash (2022),
<https://home.treasury.gov/news/press-releases/jy0916>
34. UkoeHB: Reduce scan times with 1-byte-per-output ‘view tag’ (2020),
<https://github.com/monero-project/research-lab/issues/73>
35. Wahrstätter, A., Solomon, M., DiFrancesco, B., Buterin, V., Svetinovic, D.: Basesap: Modular stealth address protocol for programmable blockchains. arXiv preprint arXiv:2306.14272 (2023)
36. Wiki, B.: ECDH Addresses, https://en.bitcoin.it/wiki/ECDH_address,
[Online; accessed 13-Jun-2023]

A Parsing Time Costs

A crucial task for the recipient in our system is detecting which transactions were sent to them. The recipient must parse through all transactions, compute the stealth address using their secret key and the public salt R derived from the transaction (see Section 7), and verify whether it matches one of the output addresses. On average, this process requires 3.07 ms per transaction, based on 100 trials. This time consists of 0.31 ms to retrieve the transaction, 2.42 ms to compute the shared secret, and 0.34 ms to derive and check the stealth address. While this time is short for individual transactions, parsing a large volume of transactions leads to a substantial increase in time.

To estimate the parsing time on the Bitcoin network, we analyzed confirmed transactions from November 16, 2022, to November 16, 2023. As shown in Fig. 5, the 75th percentiles for confirmed transactions per day, week, and month are 462,392, 3,261,635, and 14,132,707, respectively. Based on these figures, the estimated parsing times would be 23.64 minutes for a day, 2.77 hours for a week, and 12.04 hours for a month.

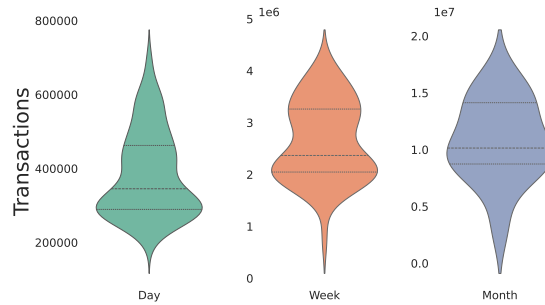


Fig. 5. Violin plot depicting the daily, weekly, and monthly confirmed Bitcoin transactions between November 16, 2022, and November 16, 2023.

One method to reduce parsing time is the use of view tags, as in Monero [34]. A view tag is a small identifier derived from the shared secret and embedded in the transaction data. This allows the recipient to quickly determine whether a transaction belongs to them without performing the full stealth address computation. This technique can reduce parsing time by 40% [23]. However, in our protocol, including view tags would reveal part of the shared secret, compromising the stealthiness of the transaction, which is a key feature of our design.

Another approach to reduce the recipient’s computational load is to use the DKSAP protocol. By sharing a secret scanning key with a server, the recipient can delegate transaction monitoring. The server continuously scans the blockchain for relevant transactions, which significantly reduces the computation required on the recipient’s side.

In some cases, the sender can also provide specific details, such as the transaction ID or date. This narrows down the range of transactions the recipient needs to check, making the parsing process faster by limiting the search to a smaller set of transactions.